

1. Write a Python program that attempts to dynamically import a module at runtime. The program should only import the module if it actually exists; otherwise, it should print "Module does not exist".
2. Create a Python package that contains two or more modules. Each module should define classes with attributes and methods. Then create another module outside the package, import the package modules, and create a subclass that inherits from at least one of the classes. Finally, create objects of both parent and child classes.
3. Create two Python modules that import each other. Run the program to observe what happens with circular imports. Then think of different ways to prevent a circular-import crash.
4. Create a package with a module containing an abstract base class (ABC). Another module in the same package should define concrete subclasses that implement all abstract methods. Write a driver program that imports these classes and demonstrates polymorphism.
5. Create three modules:  
Module A: class Animal  
Module B: class Walkable  
Module C: class Dog that inherits from both Animal and Walkable  
Demonstrate method resolution order (MRO) by calling overridden methods and printing the MRO.
6. Create a class in a module that uses private attributes and @property / @setter decorators. Import the class into another module and show how encapsulation protects the data while still allowing controlled access.
7. Create a module containing two classes where one uses composition and another uses inheritance to reuse code from a base class. Import and demonstrate the difference between the two approaches.